FINAL YEAR PROJECT

AI-Based Smart University Timetable Generator Using Constraint Optimization

A web-based system for automated, conflict-aware academic scheduling

SUBMITTED BY	ROLL NUMBER
Abdullah Mazhar	**f23binft1m01208**
SUPERVISOR	INSTITUTE
Musarat Kareem	**The Islamia University of Bahawalpur**

Departmental submission • Academic project proposal

PROJECT OVERVIEW

Project Title	AI-Based Smart University Timetable Generator Using Constraint Optimization
Project Type	Web-based academic scheduling system
Primary Focus	Constraint satisfaction, scheduling optimization, and conflict prevention

1. Introduction

University timetable scheduling is an important administrative task that involves assigning courses, instructors, classrooms, student groups, and time slots while satisfying academic and institutional requirements. Manual preparation can be time-consuming and may produce conflicts, such as assigning one instructor to two classes at the same time or allocating one classroom to multiple courses simultaneously.

This project proposes a web-based Smart University Timetable Generator that uses constraint-based optimization techniques to automatically generate feasible university timetables. The system will consider instructor availability, classroom capacity, course requirements, student groups, and available time slots. It will also attempt to improve timetable quality by reducing unnecessary gaps and balancing the distribution of classes.

2. Problem Statement

Preparing a university timetable manually requires coordination between departments, instructors, classrooms, and student groups. Managing these constraints becomes increasingly difficult as the number of courses, instructors, and rooms increases.

Manual scheduling may lead to overlapping classes, inefficient classroom utilization, unbalanced teaching schedules, and repeated revisions. Therefore, an automated timetable generation system is needed to generate conflict-free schedules, validate scheduling constraints, and assist administrators in producing more efficient university timetables.

3. Project Objectives

- Develop a web-based university timetable generation system.
- Maintain records of courses, instructors, classrooms, student groups, and available time slots.
- Generate timetables automatically using constraint-based optimization algorithms.
- Prevent instructor, classroom, and student-group scheduling conflicts.
- Consider instructor availability and classroom capacity.
- Minimize unnecessary gaps and improve timetable distribution where possible.
- Provide an interface for administrators to review, edit, and regenerate timetables.
- Allow generated timetables to be viewed, printed, or exported.
- Evaluate the system using constraint satisfaction, conflict counts, generation time, and timetable quality measures.

4. Proposed Methodology

The project will follow an iterative Software Development Life Cycle (SDLC), allowing requirements, implementation, and testing to be refined throughout development.

Phase 1 — Requirements Analysis

Identify university scheduling requirements, timetable rules, user roles, courses, instructors, rooms, and time slots.

Phase 2 — System Design

Prepare the system architecture, use-case diagram, activity diagram, database schema, and timetable scheduling workflow.

Phase 3 — Data Management

Develop modules to add, update, delete, and manage courses, instructors, classrooms, student groups, and availability information.

Phase 4 — Constraint Modeling

Hard constraints are mandatory rules that must not be violated:

- An instructor cannot teach two classes simultaneously.
- A classroom cannot host two classes simultaneously.
- A student group cannot attend two classes simultaneously.
- Room capacity must satisfy the class's required capacity.
- Instructor availability must be respected.

Soft constraints are preferences the system should optimize where feasible:

- Reduce gaps between classes.
- Distribute classes across the week.
- Avoid excessive consecutive teaching sessions.
- Respect preferred teaching periods when feasible.

Phase 5 — Timetable Generation

Implement a constraint-based scheduling algorithm. A greedy algorithm with backtracking or a suitable constraint optimization solver will be considered for generating feasible timetables. An optimization objective will be used to improve timetable quality without violating hard constraints.

Phase 6 — Web Application Development

Develop the frontend, backend, database integration, administrator dashboard, timetable display, and export functionality.

Phase 7 — Testing and Evaluation

Test scheduling conflicts, constraint satisfaction, invalid inputs, system performance, and timetable quality using sample university data.

Phase 8 — Documentation and Deployment

Prepare the final report, user guide, presentation, and a working demonstration.

5. Proposed System Modules

- Administrator Authentication and User Management
- Course and Subject Management
- Instructor Management
- Classroom and Capacity Management

- Student Group and Section Management
- Instructor Availability Management
- Time Slot and Academic Schedule Management
- Automatic Timetable Generation Engine
- Conflict Detection and Validation
- Timetable Editing and Regeneration
- Timetable Viewing and Printing
- PDF or CSV Export
- Scheduling Statistics and Reports

6. Proposed Technology Stack

Component	Proposed Technology
Programming Language	Python
Backend Framework	Flask
Frontend	HTML, CSS, JavaScript, Bootstrap
Database	SQLite initially; PostgreSQL may be considered if required
Scheduling Algorithm	Constraint-based optimization using backtracking or a suitable constraint solver
Data Processing	Pandas, if needed
Development Environment	Visual Studio Code
Version Control	Git

The initial implementation will prioritize free and open-source tools. The final algorithm will be selected according to project requirements and the university's expectations.

7. Functional Requirements

- Allow authorized administrators to log in.
- Allow administrators to manage courses, instructors, rooms, student groups, and time slots.
- Accept scheduling rules and availability information.
- Generate timetables automatically.
- Detect and prevent hard-constraint violations.
- Display scheduling conflicts or unsatisfied constraints when a feasible timetable cannot be generated.
- Allow authorized users to review and modify generated timetables.
- Export and print timetable reports.

8. Non-Functional Requirements

- **Usability:** The interface should be understandable and easy to navigate.
- **Performance:** Timetable generation should complete within a reasonable time for the supported dataset size.
- **Security:** Authentication and role-based access should protect administrative functions.

- **Reliability:** The system should validate scheduling constraints before publishing a timetable.
- **Maintainability:** The software should use modular code and clear documentation.
- **Data Integrity:** Database records should remain consistent when courses, rooms, or instructors are updated.

9. Expected Outcomes

The expected outcome is a functional web application capable of generating university timetables automatically from structured academic data. The system is expected to:

- Reduce manual scheduling effort.
- Prevent timetable clashes involving instructors, rooms, and student groups.
- Improve the use of available classrooms and time slots.
- Support timetable revision and regeneration.
- Provide printable and exportable schedules.
- Report relevant scheduling statistics and optimization results.

Actual performance improvements will be measured during testing rather than assumed in advance.

10. Scope and Limitations

The initial version will focus on timetable generation for a single department or a small university dataset. It will support courses, instructors, classrooms, student groups, availability, and a defined set of scheduling rules.

The quality of generated timetables will depend on the accuracy of the supplied data and the feasibility of the scheduling constraints. If the rules are mutually incompatible, a valid timetable may not exist. The system will report such cases rather than guarantee a solution.

Large-scale, multi-campus scheduling and integration with existing university management systems may be considered future enhancements.

11. Significance of the Project

This project applies software engineering, database management, algorithm design, and optimization to a practical university problem. It can help academic administrators prepare timetables more systematically and provide students with a clearer scheduling process.

The project will also demonstrate the practical application of constraint satisfaction and optimization in a real-world information system.

12. Conclusion

The proposed Smart University Timetable Generator aims to automate university scheduling through a web-based system supported by constraint-based optimization. By managing academic resources, enforcing essential scheduling rules, and optimizing timetable preferences, the system will provide a practical solution to common timetable preparation challenges.

The completed project will include the working application, database, scheduling algorithm, testing results, documentation, and final presentation.

Submitted By	Supervisor
Abdullah Mazhar	Musarat Kareem

Signature: _____	Signature: _____